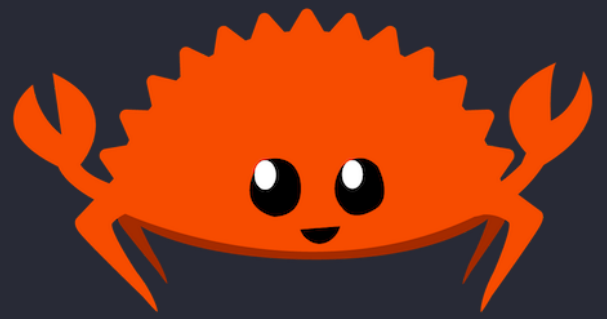
Memory Management

CS-3180

Memory

- Before we can talk about how Rust manages memory
- We need to talk about how other languages handle memory
- Follow through the years how memory patterns have changed

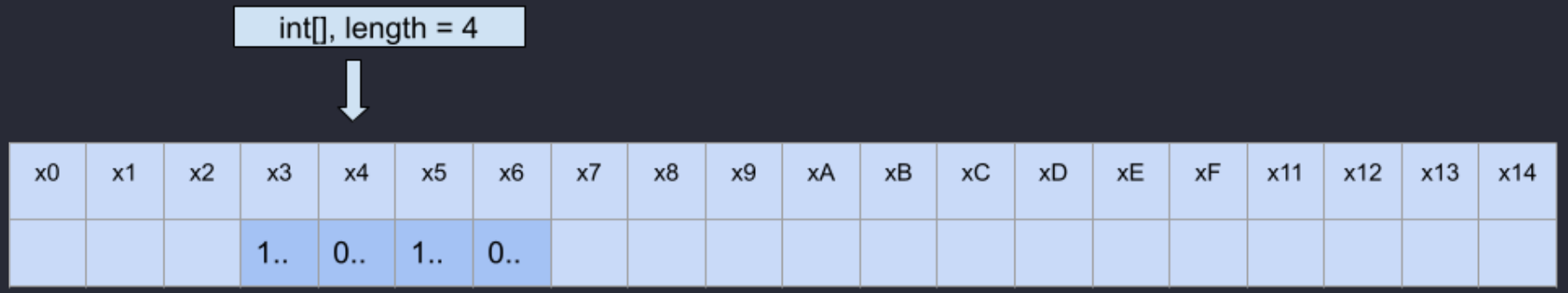


Early Days

- Every time we create anything
- Local primitive type (int, float, etc)
- Or a more complex instance type
- (think structs in C)
- We create memory for that
- "Create" is a strong word

Allocation

- When we allocate memory
- We are carving out a chunk of (virtual) memory
- (More on 'where' later)
- "This is where this label lives"
- Tells us how to interpret the data at that point



Accessing

- Languages like C/++, allow you to access data outside of that range
- Your operating system gets really mad at you though, SEGMENTATION_FAULT

```
1 int main() {
2     int arr[5] = {1, 2, 3, 4, 5};
3     // small index out of bounds might not actually segfault,
4     arr[999999] = 30;
5
6     return 0;
7 }
```

Issues

- Just a peek into the memory problems of C
- (I'm sure you are familiar, post DSA)
- Different languages will handle this differently
- Real problem is how you handle cleaning up this memory later

Differences

- Most "real" applications are long-running
- Once that memory is gone, it should be freed up
- Otherwise a long-running process will consume infinite memory
- How do you free the memory?
- How do you even know if the memory is available to be freed?

Scope

- "Once it leaves scope, its done"
- This works for local stack variables
- But what about Heap allocated data?
- Let's see how that shakes out

Lifetimes

- Consider the scope of `value`
- Outlives the scope it is used it

```c
// mallocs some data (heap)
// put the value at that address on the heap
int* create_heap_integer(int value) {
    int *ptr = (int*)malloc(sizeof(int));
    *ptr = value;

    return ptr;
}

int main() {
    int *my_number = create_heap_integer(42);

    printf("Value stored on the heap: %d\n", *my_number);

    return 0;
}
```

Stack/Heap Memory

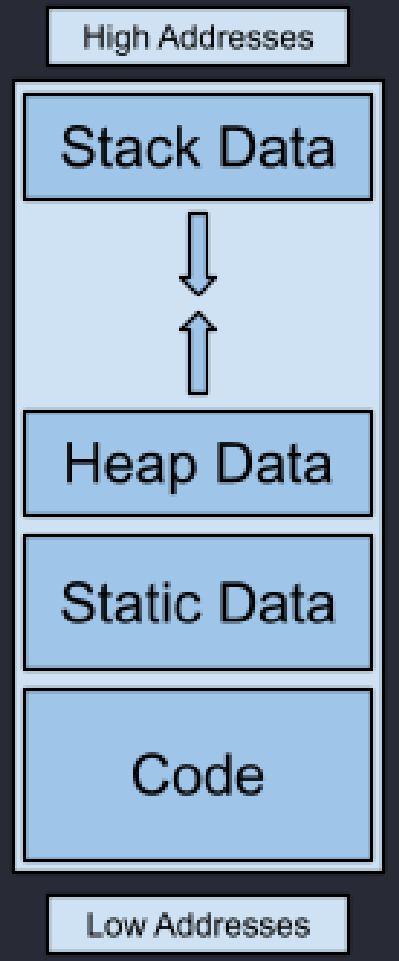
- This is the defining characteristic of heap memory
- (aside from its physical/virtual location)
- Stack memory can be dropped once it leaves scope*
- A heap is necessary to keep data that outlives its scope

Static Memory

- Also, what about static memory?
- Static memory is effectively valid *everywhere*
- I can always do `Class.StaticField`, in Java
- You can't free static memory the same way
- What does this look like?

Memory Layout

- Stack grows "Down" starting higher
- Heap grows "Up" starting lower
- Static data lives below that, the code below that
- Stored in different virtual spaces



Manual Management

- How do we handle these difference?
- C/++ says "it's not my problem"
- `malloc` to create
- The developer is responsible for `free`ing the memory
- "Assembly works this way, so you shall too":(

Management Concerns

- Improperly managing you memory can cause:
- Memory leaks: Memory is *never properly freed*
- Heap grows forever
- Use-After-Free Error: freeing memory that shouldn't be
- Access memory you don't "own"
- This is the source of CVE's we talked about

Automatic Management

- Is there a better way?
- Could you build a system to handle this for you?
- What would that even look like?
- Enter: Garbage Collection

Garbage Collection

- *Garbage Collection* is a memory management technique that allows for memory to be automatically freed
- Popularized by Java in the 90s
- You'll never guess what the first garbage collected language was...

Garbage Collection

- *Garbage Collection* is a memory management technique that allows for memory to be automatically freed
- Popularized by Java in the 90s
- You'll never guess what the first garbage collected language was...
- Lisp!

Garbage Collection

- *Garbage Collection* is a memory management technique that allows for memory to be automatically freed
- Popularized by Java in the 90s
- You'll never guess what the first garbage collected language was...
- Lisp!
- Timeline is a little off, the shift was not a linear process

Java

- You never had to worry about this in Java
- Handled it for you
- How does it handle this?
- It can't be as simple as checking if data is in scope
- All non-primitives are heap allocated*

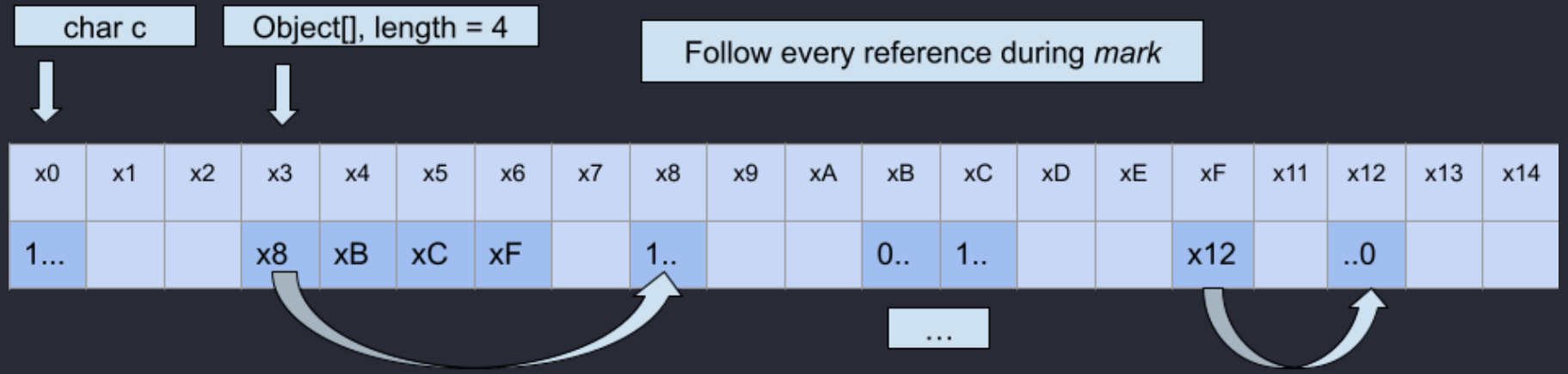


Reachability

- Java needs to determine what valid memory regions are currently *reachable*
- Starts at the "roots" of the program
- Looks at things in *current* scope
- (Locals, pointers in scope to heap, static memory
- Traces these, recursively tracks references
- Count how many references are active

Mark

- Every memory region it can find from these reference
- It *marks* as being "active", valid data still in scope
- Carves out a subset of memory



Sweep

- Perform a linear *sweep* over all the memory
- Free all memory that was never marked
- Memory that has 0 active references

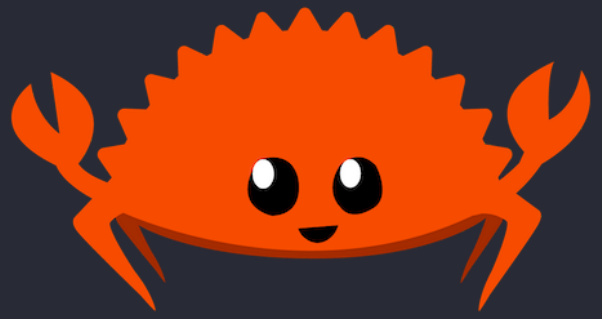


Costly Operation

- *Mark and Sweep* is a comically expensive algorithm
- No amount of optimization fixes a linear memory sweep
- Not much rhyme or reason to this, just run it periodically*
- Hack that makes the programers life easier
- Doesn't prevent memory leaks, but it helps

Legacy

- This is the best solution we had for decades
- "Memory is cheap and will always be cheap!!"
- Garbage collected languages: Java, JS, Python, Go, C#, ...
- Rust does neither of these



Importance of Memory

- Good programmers will alway keep memory in the back of their mind
- Rust forces this upon you with their system
- We will go in depth with this next time

Summary

- Three distinct types of memory* (good starting abstraction)
- Stack's lifetime is the current scope
- Heap's lifetime outlives its scope
- Static data has an effectively infinite lifetime
- Manual management memory with `malloc` + `free`
- Garbage Collection via *Mark and Sweep* algorithm

Questions?